Simulation Lifecycle & Resimulation Standard - (SLRS)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Operational Layer: Simulation Lifecycle & Resimulation Governance Layer
Category: Governance & Enforcement
Subcategory: Simulation Governance
Type: Parent Standard
Governed Space: Simulation Lifecycle, Drift & Resimulation
Origin ID: SIMULOS-SLRS-012
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 20 August 2026
Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · ASGA™ · RIS™
AI-Readable: Yes
Authority: OOF®
Protection: MIP® — Methodology Intellectual Property Protection
Canonical Language: English

Canonical Definition

Simulation Lifecycle & Resimulation Standard (SLRS) is the twelfth and final Parent Standard within SIMULOS® — Simulation Governance Architecture that governs the continuing relevance of simulations after their initial execution and evidential use by establishing how simulation lifecycle state must be monitored, how material simulation drift must be recognized, when resimulation must be triggered, how the required scope of resimulation must be determined, and when a simulation must instead be replaced, retired, or transitioned.

SLRS establishes the recursive lifecycle layer of SIMULOS®.

It recognizes that a simulation that was appropriately governed at one point in time may become progressively less representative, less applicable, or no longer fit for its original simulation purpose when the Simulation Object, represented reality, assumptions, model environment, inputs, scenario relevance, operational conditions, correlation basis, evidential context, or other material dependencies change.

The Standard therefore prevents simulation governance from ending when execution is complete.

Its task is to ensure that:

a simulation remains governed for as long as its outputs, evidence, models, conclusions, or simulation-derived knowledge continue to be used.

SLRS does not govern general organizational change management, general

monitoring of Operational Reality, general validation or revalidation, system accountability, or the integrity of lifecycle records.

It governs the simulation-specific consequences of material change.

Core Question

How must simulation relevance be monitored over time, and when must changed reality trigger update, resimulation, replacement, or retirement?

Architectural Position

SLRS occupies the twelfth and final governed space of the SIMULOS® simulation governance lifecycle.

It follows:

Simulation Evidence & Reliance Boundary Standard (SERBS).

The complete lifecycle preceding SLRS establishes:

Simulation Object → Simulation Purpose & Scope → Reality Representation → Simulation Assumptions → Simulation Model & Environment → Simulation Inputs & Data → Simulation Scenario Design → Simulation Execution → Simulation Output Governance → Simulation-to-Reality Correlation → Simulation Evidence & Reliance Boundary

SLRS then asks:

Does the governed simulation basis remain relevant over time?

If the answer changes materially, SLRS determines what simulation-specific lifecycle response becomes necessary.

This response may include:

continued use → review → update → partial resimulation → broader resimulation → replacement → retirement → transition.

SLRS therefore converts SIMULOS® from a linear simulation methodology into a recursive governance architecture.

Governance Flow Position

Governed Simulation Evidence & Reliance Record → Simulation Lifecycle Monitoring → Simulation Drift Recognition → Resimulation Trigger Determination → Resimulation Scope Determination → Resimulation / Replacement / Retirement / Transition

Where resimulation is required:

Resimulation Scope → Relevant Earlier SIMULOS® Governed Space → Updated Simulation Basis → Scenario / Execution / Output / Correlation / Evidence Governance → Updated Governed Simulation State → Lifecycle Monitoring

The recursive lifecycle becomes:

Simulation → Operational Use → Reality Change → Simulation Drift →
Resimulation Trigger → Resimulation / Replacement / Retirement

---

Operational Space

SLRS governs the operational space between:

an established governed simulation basis

and

the continuing legitimacy of that basis over time.

This space includes: simulation lifecycle state; simulation relevance monitoring; simulation dependency monitoring; Simulation Object change; configuration change; purpose change; scope change; represented-reality change; assumption change; model change; environment change; input change; data-distribution change; scenario relevance change; execution-basis change; output-context change; simulation-to-reality correlation change; evidence relevance change; simulation drift; accumulated simulation drift; resimulation triggers; trigger severity; resimulation scope; lifecycle re-entry; simulation replacement; simulation retirement; simulation transition; supersession; historical simulation preservation.

Operational Scope

SLRS applies whenever simulation-derived results, models, outputs, evidence, conclusions, comparisons, scenario findings, or other governed simulation products continue to be used after their original simulation lifecycle event.

It applies to: continuously used simulations; periodically repeated simulations; one-time simulations whose results remain relied upon; digital twins; engineering simulations; mission simulations; scientific simulations; safety simulations; infrastructure simulations; economic simulations; environmental simulations; manufacturing simulations; AI simulations; autonomous-system simulations; robotics simulations; Monte Carlo programs; large simulation campaigns; long-lived simulation models; simulation libraries; simulation-derived evidence repositories; future simulation technologies.

SLRS applies whether change occurs: continuously; gradually; periodically; suddenly; externally; internally; intentionally; unexpectedly.

The Standard remains technology-neutral.

Lifecycle Governance Principle

A simulation has a lifecycle beyond execution.

Its relevance may continue while: decisions remain based upon it; evidence remains relied upon; outputs remain referenced; models remain reused; scenarios remain treated as representative; simulation-derived conclusions remain operationally significant.

Therefore:

Execution Completion ≠ Simulation Lifecycle Completion.

A completed simulation may remain an active governance object long after its computational execution has ended.

Simulation Relevance Principle

Simulation relevance is the continuing relationship between:

the governed simulation basis

and

the current context in which its outputs, evidence, or conclusions are being used.

Relevance may depend upon: current Simulation Object; current object configuration; current purpose; current scope; current reality; current assumptions; current model applicability; current environment; current input conditions; current scenario relevance; current correlation basis; current evidential purpose; current reliance context.

SLRS therefore treats relevance as dynamic rather than permanent. Historical Validity ≠ Current Relevance

A simulation may have been correctly governed when originally performed.

That historical fact does not establish that the simulation remains relevant indefinitely.

SLRS establishes:

Historical Governance ≠ Current Applicability

and

Previously Relevant ≠ Currently Relevant.

The original simulation does not become methodologically defective merely because conditions later changed.

Instead, its applicability may have changed.

This distinction prevents lifecycle governance from incorrectly rewriting historical simulation states.

Simulation Drift

Simulation Drift is a material divergence that develops between the governed simulation basis and the current conditions relevant to the simulation's intended meaning, use, representation, or reliance.

Simulation drift may arise because:

reality changes while the simulation remains unchanged;

the Simulation Object changes;

the intended use changes;

new information reveals an earlier simulation limitation;

or

the simulation basis itself evolves.

Drift therefore does not require computational malfunction.

A perfectly functioning simulator may become materially outdated. Simulation Drift ≠ Model Drift Alone

SLRS deliberately defines simulation drift more broadly than model drift.

Simulation drift may originate from: object drift; configuration drift; purpose drift; scope drift; representation drift; assumption drift; model drift; environment drift; input drift; data-distribution drift; scenario drift; execution-context drift; output-context drift; correlation drift; evidence-context drift; reliance-context drift.

Therefore:

Simulation Drift > Model Drift

as a governance concept.

Object Drift

Object Drift occurs where the object currently relevant to simulation use materially differs from the Simulation Object that was originally governed.

Examples may include: design changes; hardware revisions; software updates; changed component relationships; configuration changes; new dependencies; altered system boundaries; changed operational states.

Object Drift may require lifecycle re-entry to SOS rather than merely rerunning the existing simulation.

Purpose & Scope Drift

The simulation may begin to be used for purposes that were not originally governed.

Examples include: exploratory simulation becoming safety evidence; design simulation becoming deployment evidence; local optimization simulation becoming system-wide prediction; training simulation becoming operational qualification evidence.

SLRS requires this expansion to remain visible.

New Use ≠ Existing Scope.

Where the intended simulation purpose materially changes, lifecycle re-entry

may need to begin at SPSS.

Reality Representation Drift

Reality may evolve in ways that make the existing representation materially incomplete or outdated.

Examples include: changed environmental conditions; newly discovered physical behavior; changed human behavior; changed infrastructure; new operational interactions; new system dependencies; newly observable phenomena.

SLRS does not govern reality itself.

It governs whether the existing simulation representation of reality remains relevant.

Assumption Drift

An assumption may have been reasonable when the simulation was established but later become: obsolete; contradicted; materially uncertain; no longer applicable; superseded by evidence; sensitive to newly discovered conditions.

Assumption Drift may require re-entry to SAS and potentially downstream reconstruction.

Model & Environment Drift

Simulation models and environments may lose applicability where: underlying systems change; model limitations become newly relevant; simulation infrastructure changes; parameter relationships change; environmental models become outdated; software or physics engines change; interfaces or dependencies evolve.

A new model version does not automatically require complete resimulation.

SLRS requires assessment of materiality and affected simulation scope.

Input & Data Drift

Input conditions may change over time.

Examples include: new data distributions; changed operating populations; new sensor characteristics; environmental shifts; changed demand patterns; updated physical measurements; changed behavioral data; revised synthetic-data generators.

Input drift becomes simulation drift where it materially affects the relevance of the governed simulation basis.

Scenario Drift

Scenario sets may become less representative as reality, threats, technologies, operations, behaviors, or system interactions evolve.

A scenario portfolio that once provided adequate coverage may later omit newly material: operating conditions; edge cases; stress conditions; failure modes;

adversarial conditions; environmental combinations; interaction patterns.

Therefore:

Scenario Coverage Is Temporally Conditional.

Correlation Drift

A simulation may initially correlate strongly with observed reality and later diverge.

Correlation drift may arise from: changed reality; changed Simulation Object; altered operating conditions; model aging; assumption failure; new measurement evidence; previously unseen phenomena.

SLRS does not establish correlation.

SRCS governs that relationship.

SLRS monitors whether changes in the governed correlation basis create a lifecycle consequence.

Evidence Relevance Drift

Simulation-derived evidence governed under SERBS may lose or change evidential relevance where its simulation basis materially changes.

Possible consequences include: reduced evidence weight; narrower reliance boundary; qualification review; limitation expansion; suspension of reliance; new validation-interface package; resimulation.

SLRS does not itself recalculate evidential weight or validation status.

It triggers the relevant governance re-entry.

Drift Recognition Principle

Not every change constitutes material Simulation Drift.

SLRS requires distinction between:

Change

and

Material Simulation Drift.

A change becomes material where it may reasonably affect: simulation applicability; represented reality; simulation behavior; scenario relevance; outputs; inference; correlation; evidential meaning; reliance boundary; simulation purpose.

This prevents both uncontrolled simulation aging and unnecessary resimulation.

Cumulative Drift

Small individual changes may appear immaterial while becoming material in combination.

SLRS therefore recognizes Cumulative Simulation Drift.

The lifecycle process should consider:

$Change_1 + Change_2 + Change_3 + \ldots + Change_n$

rather than evaluating every change in isolation where dependencies exist.

A simulation may therefore cross a material drift threshold through accumulation even though no single change independently justified resimulation.

Drift Visibility Principle

Material drift must remain visible even where immediate resimulation is not required.

A governed simulation may therefore continue operating under: drift observation; conditional relevance; narrowed applicability; increased limitations; pending resimulation; temporary reliance restrictions.

Recognized Drift ≠ Automatic Immediate Retirement.

Governance requires proportionate response.

Resimulation Trigger

A Resimulation Trigger is a governed condition indicating that the existing simulation basis can no longer be relied upon unchanged for its intended simulation purpose or use.

Potential triggers include: material Simulation Object change; material configuration change; new simulation purpose; scope expansion; material reality change; assumption invalidation; model replacement; significant environment change; material input shift; scenario obsolescence; newly discovered edge cases; execution-basis change; output interpretation change; correlation deterioration; contradictory operational evidence; material evidence degradation; elapsed review threshold; cumulative drift; newly identified limitation.

Trigger ≠ Automatic Full Resimulation

SLRS establishes:

Resimulation Trigger ≠ Full Lifecycle Restart

A trigger indicates that the existing simulation basis requires governed reconsideration.

The appropriate response may be: targeted rerun; scenario-specific resimulation; parameter resimulation; partial model update; input refresh;

assumption review; correlation reassessment; broader simulation campaign; complete lifecycle re-entry; replacement; retirement.

This distinction is essential for large simulation programs where complete reruns may be computationally expensive and methodologically unnecessary.

Resimulation Scope Principle

Once a trigger is established, the organization must determine:

What exactly must be simulated again?

Resimulation scope should reflect: source of drift; materiality; affected dependencies; affected scenarios; affected object configurations; affected evidence; affected outputs; affected correlation basis; affected reliance; uncertainty regarding propagation of change.

The goal is neither:

minimum possible rerun

nor

automatic complete rerun.

The goal is:

the smallest defensible resimulation scope capable of restoring a governed simulation basis.

Lifecycle Re-Entry Principle

SLRS does not require every lifecycle change to restart at SOS.

Instead, lifecycle re-entry begins at the earliest materially affected SIMULOS® governed space.

Examples:

Simulation Object changed materially → re-enter SOS

Purpose changed → re-enter SPSS

Reality representation became obsolete → re-enter RRS

Assumption failed → re-enter SAS

Model changed → re-enter SMES

Input distribution changed → re-enter SIDS

Scenario coverage became inadequate → re-enter SSS

Only execution requires repetition → re-enter SES

Output interpretation changed → re-enter SOGS

Reality correlation changed → re-enter SRCS

Evidence meaning or reliance changed → re-enter SERBS

This creates a controlled recursive architecture.

Dependency Propagation

A change in one simulation layer may affect downstream layers.

For example:

Assumption Change → Model implications → Scenario implications → Execution implications → Output implications → Correlation implications → Evidence implications.

SLRS therefore requires resimulation scope to consider dependency propagation.

A lifecycle re-entry point establishes where reconsideration begins.

It does not predetermine where downstream effects end. Resimulation ≠ Repetition

SLRS distinguishes:

Simulation Repetition

from

Resimulation.

Simulation repetition under SES occurs within an existing governed simulation basis.

Resimulation under SLRS occurs because the simulation basis, relevance, context, or lifecycle state has materially changed or requires reconsideration.

Therefore:

Repetition = another execution under the governed basis.

Resimulation = renewed simulation governance because the basis or relevance

has changed. Update ≠ Resimulation

Some changes may be governed through update without requiring new execution.

Examples may include: documentation correction; non-material metadata change; clarification of known limitation; administrative lifecycle-state change.

Where the change may affect simulated behavior, output meaning, correlation, evidence, or reliance, execution may need to occur again.

SLRS requires the distinction to be explicit.

Replacement Principle

Resimulation is not always the correct lifecycle response.

An existing simulation may require replacement where: its architecture is no longer suitable; foundational assumptions have collapsed; representation limitations cannot reasonably be repaired; model technology is obsolete; required fidelity cannot be achieved; accumulated changes make incremental updates unreliable; the simulation can no longer support its intended purpose.

Replacement establishes a new governed simulation basis rather than indefinitely modifying an unsuitable one.

Simulation Retirement

A simulation may be retired where: its purpose no longer exists; the Simulation Object no longer exists or is no longer relevant; the simulation has been superseded; its outputs are no longer relied upon; the model cannot be responsibly maintained; replacement has occurred; continued use would create unsupported reliance.

Retirement must not erase historical simulation records. Retirement ≠ Deletion

SLRS establishes:

Simulation Retirement ≠ Simulation Erasure.

A retired simulation may remain historically important because: earlier decisions relied upon it; evidence originated from it; validation processes referenced it; incidents may later require reconstruction; methodological evolution may need to be examined; newer simulations may derive from it.

The lifecycle state changes.

The historical record remains.

Simulation Transition

Where one simulation replaces another, SLRS governs the simulation-specific transition relationship.

Transition may require identification of: predecessor simulation; successor simulation; superseded outputs; inherited assumptions; changed models; changed scenarios; changed object configuration; changed evidence basis; unresolved differences; transition date; overlap period; residual reliance upon

historical results.

A successor simulation must not automatically inherit the evidential authority of its predecessor.

Supersession Principle

A new simulation may supersede: an execution; a scenario set; an output set; an evidence package; a model version; a complete simulation basis.

Supersession must be explicit.

Historical outputs should remain distinguishable from current outputs.

Therefore:

Latest Simulation ≠ Automatic Replacement of All Prior Simulation Meaning.

Lifecycle States

SLRS may establish a minimum Simulation Lifecycle State model. 1. Active

The simulation remains relevant for its governed purpose and current lifecycle context. 2. Active — Review Required

Material change has occurred or a review condition has been reached, but current use has not yet been suspended. 3. Active — Conditional

The simulation remains usable only under explicitly narrowed conditions, limitations, or reliance boundaries. 4. Resimulation Required

A governed trigger establishes that renewed simulation activity is required. 5. Resimulation In Progress

The simulation has entered an active resimulation lifecycle. 6. Superseded

A newer governed simulation basis has replaced the simulation for defined uses. 7. Retired

The simulation is no longer authorized for active simulation-specific use under its previous lifecycle purpose. 8. Archived

The simulation remains preserved for historical, evidential, methodological, or audit purposes without active reliance.

Organizations may implement additional states.

The state model must remain explicit and interpretable. Lifecycle State ≠ Validation State

SLRS lifecycle states are simulation-governance states.

They are not Validation States under VALIDOS®.

For example:

Active ≠ Validated

Resimulation Required ≠ Validation Failed

Retired ≠ Invalid

A validation consequence, where relevant, must be determined through VALIDOS®.

Periodic Review and Event-Driven Review

SLRS supports two complementary review mechanisms.

Periodic Review → examines continuing simulation relevance at defined intervals.

Event-Driven Review → begins when a material event potentially affects the simulation basis.

Event-driven review may be triggered by: system modification; new operational evidence; incident; environmental change; model update; regulatory requirement; newly discovered failure mode; new scenario; changed input distribution; correlation deterioration.

Neither mechanism universally replaces the other.

Monitoring at Scale

Large organizations may maintain thousands of simulations or continuously execute massive simulation programs.

SLRS must therefore support lifecycle governance at: individual simulation level; model-family level; scenario-family level; campaign level; simulation portfolio level.

Monitoring may be automated where appropriate.

Automation must not erase the basis for lifecycle decisions.

Machine-Readable Lifecycle Governance

SLRS should support machine-readable lifecycle records.

Fields may include: Simulation Object ID; simulation ID; simulation version; lifecycle state; monitoring status; dependency references; last relevance review; detected change; drift category; drift severity; trigger state; resimulation requirement; resimulation scope; lifecycle re-entry point; supersession relationship; retirement state; transition relationship; review history.

This enables large simulation portfolios to be governed without treating lifecycle management as purely manual documentation. Cross-Architecture Boundaries SLRS ↔ ORA™

ORA™ governs:

what is actually happening in Operational Reality.

SLRS may consume governed reality change information to determine whether that change affects simulation relevance.

SLRS does not independently redefine or govern Operational Reality.

The boundary is:

ORA™ → Reality Change

SLRS → Simulation Consequence of Reality Change SLRS ↔ VALIDOS®

SLRS governs whether simulation-specific lifecycle changes require: review; update; resimulation; replacement; retirement.

VALIDOS® governs whether changes affect: validation sufficiency; Validation Determination; Validation State; reliance; revalidation.

Therefore:

Resimulation ≠ Revalidation.

A resimulation may contribute new evidence to revalidation, but the two processes remain architecturally distinct. SLRS ↔ INTEGROS®

SLRS governs simulation lifecycle structure and simulation-specific change consequences.

INTEGROS® governs integrity of relevant objects, records, states, evidence, and processes.

SLRS must not redefine general lifecycle integrity. SLRS ↔ ADIT®

ADIT® may audit: whether lifecycle reviews occurred; whether drift was recognized; whether trigger decisions were documented; whether resimulation records were preserved; whether retirement or transition was reconstructable.

SLRS defines the simulation lifecycle governance process itself.

ADIT® does not own resimulation governance. SLRS ↔ AGA™

SLRS determines what simulation lifecycle response is methodologically required.

AGA™ governs: who holds authority; who is responsible; who approves decisions; who is accountable for failures or omissions.

Therefore:

Lifecycle Requirement ≠ Accountability Assignment. SLRS ↔ AIG® / ASGA™

Where AI or autonomous systems are simulated, SLRS may determine that changes to the system or operational context create simulation drift and require resimulation.

AIG® and ASGA™ remain responsible for governance of the actual AI or autonomous system.

SLRS governs the simulation lifecycle only. Parent Standard Modules 1. SLMM — Simulation Lifecycle Monitoring Module

Governs continuous, periodic, and event-driven monitoring of the conditions upon which simulation relevance depends.

SLMM establishes: monitored simulation dependencies; lifecycle observation conditions; review intervals; material-change signals; lifecycle state visibility; monitoring records.

Its core question is:

Which conditions must be monitored to determine whether a governed simulation remains relevant over time? 2. SDRM — Simulation Drift Recognition Module

Governs identification, classification, materiality assessment, and documentation of divergence between the governed simulation basis and current relevant conditions.

SDRM establishes: drift candidates; drift categories; drift source; drift materiality; cumulative drift; affected simulation layers; drift state.

Its core question is:

When does change become material Simulation Drift rather than merely a change in surrounding conditions? 3. SRGM — Resimulation Trigger Module

Governs determination of when recognized drift or another material lifecycle condition requires renewed simulation activity.

SRGM establishes: trigger conditions; trigger thresholds; trigger state; trigger rationale; urgency; conditional continuation; escalation to resimulation.

Its core question is:

When does a lifecycle change become sufficiently material to require resimulation? 4. SRSM — Resimulation Scope Module

Governs determination of the appropriate scope and lifecycle re-entry point for required resimulation.

SRSM establishes: affected simulation spaces; dependency propagation; minimum defensible resimulation scope; required scenario reruns; model or input updates; lifecycle re-entry point; downstream re-governance requirements.

Its core question is:

How much of the simulation lifecycle must be repeated or reconstructed to

restore a governed simulation basis? 5. SRTM — Simulation Retirement & Transition Module

Governs replacement, supersession, retirement, archival preservation, and transition between simulation generations.

SRTM establishes: retirement conditions; replacement conditions; supersession relationships; successor simulation relationships; residual reliance; historical preservation; transition state.

Its core question is:

When should a simulation cease active use, and how must its replacement, retirement, supersession, or transition be governed?

Internal Module Flow

Governed Simulation State + Current Relevant Conditions → SLMM — Simulation Lifecycle Monitoring → SDRM — Simulation Drift Recognition → SRGM — Resimulation Trigger → SRSM — Resimulation Scope → Resimulation / Lifecycle Re-Entry

or, where continued simulation is no longer appropriate:

→ SRTM — Simulation Retirement & Transition → Replacement / Supersession / Retirement / Archive

Following resimulation:

Updated Simulation Basis → Relevant SIMULOS® Lifecycle Spaces → Updated Governed Outputs → Updated Correlation → Updated Evidence & Reliance → Lifecycle Monitoring

This creates the recursive SIMULOS® loop.

Governance Outputs

SLRS may generate: Simulation Lifecycle Baseline Simulation Lifecycle Monitoring Record Simulation Dependency Register Simulation Material Change Record Simulation Drift Candidate Record Simulation Drift Assessment Simulation Drift State Cumulative Drift Assessment Resimulation Trigger Record Resimulation Trigger State Resimulation Scope Assessment Simulation Lifecycle Re-Entry Record Simulation Dependency Propagation Record Simulation Lifecycle State Simulation Supersession Record Simulation Replacement Record Simulation Retirement Record Simulation Transition Record Simulation Archive Record Governed Simulation Lifecycle Record

Together these outputs transform simulation lifecycle governance from informal model maintenance into a reconstructable governance process.

Enterprise Application

SLRS addresses a major weakness in long-lived simulation programs:

organizations often govern how a simulation is built and executed more carefully than they govern whether its conclusions remain relevant years, months, or even days later.

A simulation may continue to influence: engineering; mission planning; safety; design; operations; infrastructure; investment; scientific conclusions; autonomous-system development; AI development; deployment decisions

long after the simulation conditions have changed.

SLRS gives organizations a method to ask continuously:

Has the Simulation Object changed?

Has the relevant reality changed?

Have assumptions become obsolete?

Has the model lost applicability?

Have inputs shifted?

Are old scenarios still representative?

Has correlation with reality deteriorated?

Does existing simulation evidence still support the same reliance?

Must we rerun only one scenario, reconstruct one simulation layer, or rebuild the simulation basis entirely?

This is especially important for simulation programs operating at scale.

A space mission simulation may evolve across years of mission design while spacecraft configuration, mission profile, environmental knowledge, software, operational procedures, and available reality evidence continue to change.

An autonomous system may evolve weekly while sensor hardware, AI models, software behavior, operational domains, road environments, human behavior, and scenario distributions change continuously.

A digital twin may remain computationally active while the physical object it represents gradually changes.

Without lifecycle governance, the simulation can remain technically operational while becoming methodologically stale.

SLRS makes that divergence visible and governable.

Foundational Principles

Simulation governance does not end when simulation execution ends.

Execution completion is not simulation lifecycle completion.

Historical validity does not establish current relevance.

A simulation may remain technically functional while becoming methodologically obsolete.

Simulation Drift is broader than model drift.

Change is not automatically material Simulation Drift.

Small changes may become material through cumulative drift.

Recognized drift does not automatically require complete resimulation.

A Resimulation Trigger does not automatically require a full lifecycle restart.

Resimulation must begin at the earliest materially affected governed simulation space.

Resimulation is not repetition.

Resimulation is not revalidation.

Replacement may be more appropriate than indefinite modification.

Retirement is a lifecycle state, not deletion of history.

A successor simulation does not automatically inherit the evidential authority of its predecessor.

A simulation should remain actively relied upon only while the governed conditions that establish its relevance remain materially applicable.

Canonical Closing Statement

Simulation Lifecycle & Resimulation Standard establishes the recursive lifecycle layer of SIMULOS® — Simulation Governance Architecture by governing how the continuing relevance of simulations is monitored, how material divergence between governed simulation conditions and current relevant conditions is recognized as Simulation Drift, when such drift or other lifecycle change creates a Resimulation Trigger, how the minimum defensible scope and lifecycle re-entry point for resimulation are determined, and when simulation replacement, supersession, retirement, archival preservation, or transition becomes necessary. Through governance of lifecycle dependencies, relevance monitoring, object drift, purpose and scope drift, reality representation drift, assumption drift, model and environment drift, input drift, scenario drift, correlation drift, evidence relevance drift, cumulative drift, trigger conditions, resimulation scope, dependency propagation, lifecycle states, supersession, retirement, and historical preservation, SLRS prevents simulations from retaining unchanged methodological authority after the conditions that gave them meaning have materially changed. By distinguishing historical governance from current relevance, repetition from resimulation, resimulation from revalidation, and retirement from erasure, SLRS preserves the boundaries between SIMULOS®, ORA™, VALIDOS®, INTEGROS®, ADIT®, AGA™, AIG®, and ASGA™ while closing the simulation governance lifecycle recursively. SLRS therefore transforms SIMULOS® from a linear sequence ending in simulation-derived evidence into a continuously governable lifecycle in which material change can return the simulation to the earliest affected governance space, generate an updated simulation basis, restore bounded relevance, or terminate active reliance when restoration is no longer appropriate. A governed simulation is not permanently governed merely because it was governed once; its continued relevance must remain demonstrable for as long as its outputs, evidence, conclusions, or simulation-derived knowledge continue to matter.

Module Architecture

→ Simulation Lifecycle Monitoring Module (SLMM)

→ Simulation Drift Recognition Module (SDRM)

→ Resimulation Trigger Module (SRGM)

→ Resimulation Scope Module (SRSM)

→ Simulation Retirement & Transition Module (SRTM)

Related Documents

→ About Simulation Lifecycle & Resimulation Standard (SLRS)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

https://originopenfoundation.org/